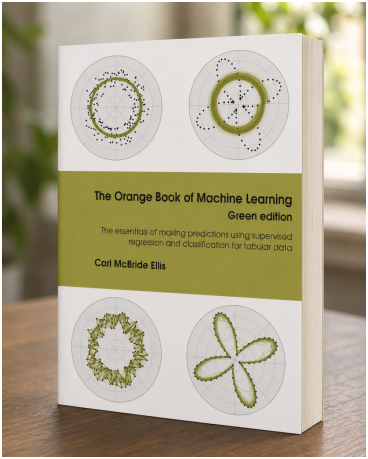
Machine learning: Intro

Carl McBride Ellis

carl.mcbride@u-tad.com

Recommended text "The Orange Book of Machine Learning - Green edition" (paperback version)



A pdf copy of this book is provided with the course.

Types of data

- unstructured: *i.e.* (photos, text: web pages, pdf, *etc.*)
- semi-structured: *i.e.* JSON format (JavaScript Object Notation)
- structured:: *i.e.* CSV format (comma-separated values)

example of a semi-structured JSON file

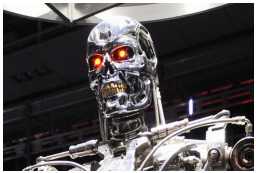
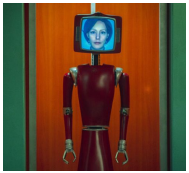
```
val json ="""
{
    "id": "0001",
    "type": "donut",
    "name": "Cake",
    "ppu": 0.55,
    "batters":
        {
            "batter":
                [
                    { "id": "1001", "type": "Regular" },
                    { "id": "1002", "type": "Chocolate" },
                    { "id": "1003", "type": "Blueberry" },
                    { "id": "1004", "type": "Devil's Food" }
                ]
        },
    "topping":
        [
            { "id": "5001", "type": "None" },
            { "id": "5002", "type": "Glazed" },
            { "id": "5005", "type": "Sugar" },
            { "id": "5007", "type": "Powdered Sugar" },
            { "id": "5006", "type": "Chocolate with Sprinkles" },
            { "id": "5003", "type": "Chocolate" },
            { "id": "5004", "type": "Maple" }
        ]
}
"""
```

example of a structured CSV file

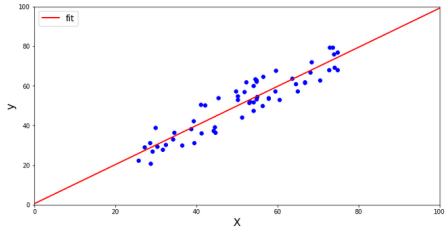
```
Age,Pclass,Sex,SibSp,Parch,Embarked,Fare
22.0,3,male,1,0,S,7.25
38.0,1,female,1,0,C,71.2833
26.0,3,female,0,0,S,7.925
35.0,1,female,1,,S,53.1
35.0,3,male,0,0,S,8.05
,3,male,0,0,Q,8.4583
54.0,1,male,0,0,S,51.8625
2.0,,male,3,1,S,21.075
27.0,3,female,0,2,S,11.1333
14.0,2,female,1,0,C,30.0708
4.0,3,female,1,1,S,16.7
58.0,1,female,0,0,S,26.55
20.0,3,male,0,0,S,8.05
39.0,3,male,1,5,S,31.275
14.0,3,female,0,0,S,7.8542
55.0,2,female,0,0,S,16.0
2.0,3,male,4,1,Q,29.125
```

- machine learning (ML) $\leftarrow$ structured data or 'tabular data'
- for machine learning **ALL** of the data must be numeric (of type `float` or `integer`)

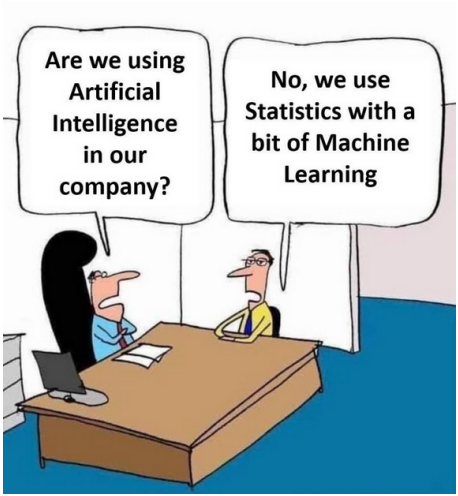
- **ML $\neq$ "artificial intelligence" (AI)**


- **ML = glorified curve fitting**






Are we using Artificial Intelligence in our company?
No, we use Statistics with a bit of Machine Learning

What is ML for?

$\hat{\beta}$: **Understand (inference)**: know which factors influence y

$\hat{y}$: **Predict**: build a predictive model for y

Artículos:

- Leo Breiman "Statistical Modeling: The Two Cultures"
- Galit Shmueli "To Explain or to Predict?"

Data science vs Data engineering

In data science we wish to understand the data generating process (DGP) and then make a mathematical model of said process. The performance metric indirectly reflects our understanding of the signal; the better the metric, the better we (assume) we are modelling the DGP. This is achieved via feedback obtained from numerical experiments.

Data engineers will then incorporate this model into a production environment. This will involve maintaining a pipeline to ensure high availability high speed predictions for very large volumes of production data, as well as the constant monitoring of the quality of said predictions for performance degradation due to data drift.



vs



Example: Credit card fraud

A data scientist may be given a dataset of a snapshot of several thousand credit card transactions, some of which are fraudulent. They must take this data and, during several several weeks or months, craft a fraud classification model.

The data engineer must design and maintain a high availability end-to-end system to take a constant real-time flow of millions of worldwide transactions, use the ML model to make a prediction, and return a ACCEPTED or DENIED decision, all in less than 0.1s

We shall use `python` + Jupyter notebooks (see also marimo notebooks)







alternatives: `R` + RStudio or Positron (warning: "*The R Inferno*")

(python in R with `reticulate`)







and also: `julia` + Pluto notebooks (free book "Julia Data Science")

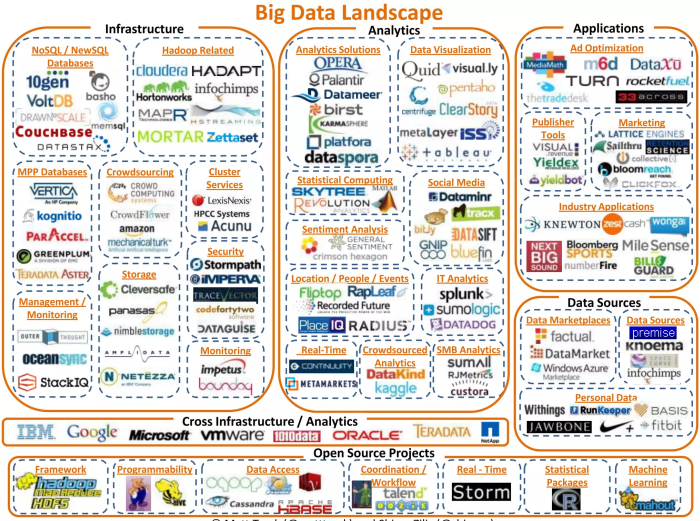




Book about Python: "The Python Workshop"

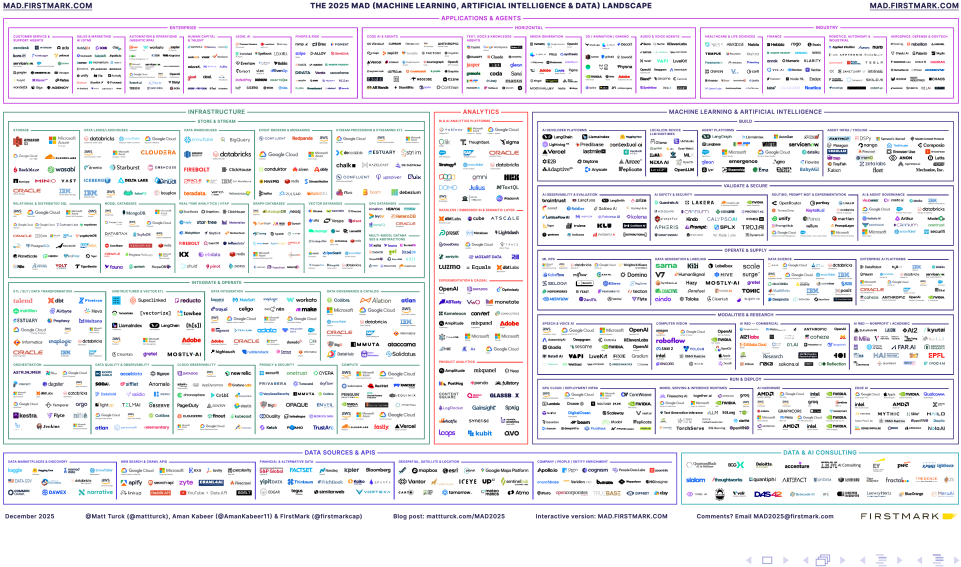


ML, AI and Data landscape 2012



© Matt Turck (@mattturck) and ShivonZilis (@shivonz)

ML, AI and Data landscape 2025



Core

```
numpy
scipy
scikit-learn    # models, transforms, metrics
matplotlib      # low-level plotting
seaborn         # higher-level plotting
```

Tabular

```
# -- dataframes --
pandas
polars

# -- gradient-boosted decision trees --
lightgbm
xgboost
catboost

autogluon       # AutoML on dataframes
```

Source: "*The State of Machine Learning Competitions*" (2025 Edition)

PyData













some other ML frameworks:

- H2O.ai
- Spark MLlib
- RAPIDS (for GPU)

cloud end-to-end service

- Vertex AI
- SageMaker
- Azure AI

Essential tools

environments:

- Jupyter notebooks: Jupyter Lab / Google Colab / Kaggle / ...

  (`pip3 install jupyterlab`)

language:

- Python (latest: octubre 2025 versión 3.14)

libraries:

- numpy (ver tb. `numba`, `numexpr`)
- pandas
- matplotlib
- scikit-learn *'Machine Learning in Python'*

(Artículo: "*Why Jupyter is data scientists' computational notebook of choice*" **Nature** (2018))

Recommended



To work with Python and Jupyter we can also use Jupyter Notebooks on Kaggle, which has many important ML packages already pre-installed.

Requirements

These are some essential Python packages that you will need to have installed for this course if you are working in local:

```
numpy
pandas
matplotlib
seabon
plotly
missingno
graphviz
statsmodels
scipy
scikit-learn
eli5
xgboost
lightgbm
catboost
```

(`pip install -r requirements.txt`)

Notebooks

Title
subtitle
sub-subtitle

```
[web link](https://www.url.es)
```

(for more see: markdown syntax)

Equations using LaTeX markup *i.e.* in-line `$ a=b+c $` and stand-alone `$$ a=b+c $$` (for more see: Beautiful math in your notebook)

(some) **Jupyter Keyboard shortcuts**

- Shift + Enter: run cell
- Esc, a: open cell above
- Esc, b: open cell below
- Esc, y: change cell to code
- Esc, m: change cell to markdown
- Esc, dd: delete cell
- Ctrl a, Ctrl /: comment/uncomment whole cell
- Ctrl a, Tab: indent whole cell
- Ctrl a, Shift + Tab: unindent whole cell
- Esc, Ctrl a, Shift + Enter: run whole notebook
- Ctrl b: toggle full-width notebook
- Ctrl s: save notebook

As a data scientist your notebooks (or `.py` scripts) will nearly always start with importing these three libraries:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```



Pandas

Data manipulation: `pandas`




```
import pandas as pd
```

(pandas 3.0.1: Feb. 2026)

("*10 Things I Hate About pandas*" by Wes McKinney himself)

Tabular data is often provided in csv format (*comma-separated values*)
We read this data using pandas read_csv, for example

```
import pandas as pd
df = pd.read_csv("mis_datos.csv")
```

In this moment a copy of the file mis_datos.csv is in the memory of your notebook, as the df dataframe object

One can also read SQL tables with pd.read_sql() and Exclel files with pd.read_excel()

(requirement: pip3 install xlrd)

A dataframe has a special row containing the names of the columns, known as the "header"

There is also a special column to label the n rows $[0, \ldots, n-1]$. This is called the "index"

Note: `Python`, `numpy` and `pandas` use zero-based numbering



some useful flags for reading

- `decimal=","` – decimal commas instead of decimal points
- `sep=";"` – if your `csv` is really a `;sv`
- `delim_whitespace=True` – by default is `False`
- `index_col="new_index"` – specify the index column
- `nrows=10_000` – just read the first 10K rows

Note: although we shall be reading `.csv` files, datasets can come in many other formats, for example:

- `JSON`/`JSONL`/`NDJSON` - para datos semi-estructurados
- `Avro`
- `ORC`
- `feather`
- `hdf5`
- `jay`
- `parquet` (guardado en columnas)
- bases de datos + SQL + `pySpark`

```
pandas.read_clipboard()
pandas.read_csv()
pandas.read_excel()
pandas.read_feather()
pandas.read_fwf()
pandas.read_gbq()
pandas.read_hdf()
pandas.read_html()
pandas.read_json()
pandas.read_orc()
pandas.read_parquet()
pandas.read_pickle()
pandas.read_sas()
pandas.read_spss()
pandas.read_sql()
pandas.read_stata()
pandas.read_table()
pandas.read_xml()
```

```
[18]  df = pd.read_excel(fil
      ✓ 91m 22.9s
```

```
[19]  df = pd.read_parquet(
      ✓ 50.4s
```

There are many, much faster, alternatives to `pandas`:

- `Modin` (`import modin.pandas as pd`)
- `dask`
- `PyArrow`
- `datatable` (last release: 2021)
- `rapids` (usando los GPU)
- `vaex`
- `polars` (first release julio 2021, latest release 1.43)

(Database-like ops benchmark)

("*Pandas Is Not Enough? A Comprehensive Guide To Alternative Data Wrangling Solutions*")

The essential object in pandas is the 'series' (`pd.Series`) which is an indexed column (*i.e.* 1D data) that has a single type (`dtype`):

```
id
0          132000
1          481000
2          927000
3          351000
4          856000
           ...
199995     388000
199996     337000
199997    1156000
199998    1118000
199999    1010000
Name: imp_val, Length: 200000, dtype: int64
```

a `pd.DataFrame` is a collection of `pd.Series` side-by-side (*i.e.* 2D data).

Slicing dataframes: `loc` y `iloc`

`loc` for 'location' based on names, and
`iloc` for 'index location' based on numerical position:

```
# select columns using names
df.loc[:, ['feature_1', 'feature_2']]
# or simply
df[['feature_1', 'feature_2']]
# select column numbers 2 and 7 using position
df.iloc[:, [2, 7]]
```

To take a random `sample` of (here 500) rows of a dataframe (df)

```
df_sample = df.sample(n=500, random_state=42)
```

To sort a df by the values in a specified column

```
df.sort_values(by='column', ascending=False)
```

To filter a df, for example, select rows for which the value of feature is between 10 and 20:

```
df_filtered = df[(df["feature"] >= 10) & (df["feature"] <= 20)]
```

To create new columns ('features') in an existing df

```
df["kk"] = 1234
df["jj"] = df["kk"]*2
```

and to delete (drop) columns

```
df = df.drop(columns=['kk', 'jj'])
# or
df = df.drop(['kk', 'jj'], axis=1) # axis 1 is columns,
                                   # axis 0 (default) is rows
```

(Note that in python and pandas `"kk"` is the same as `'kk'`)

To learn more about `pandas`:
manual 'on-line':

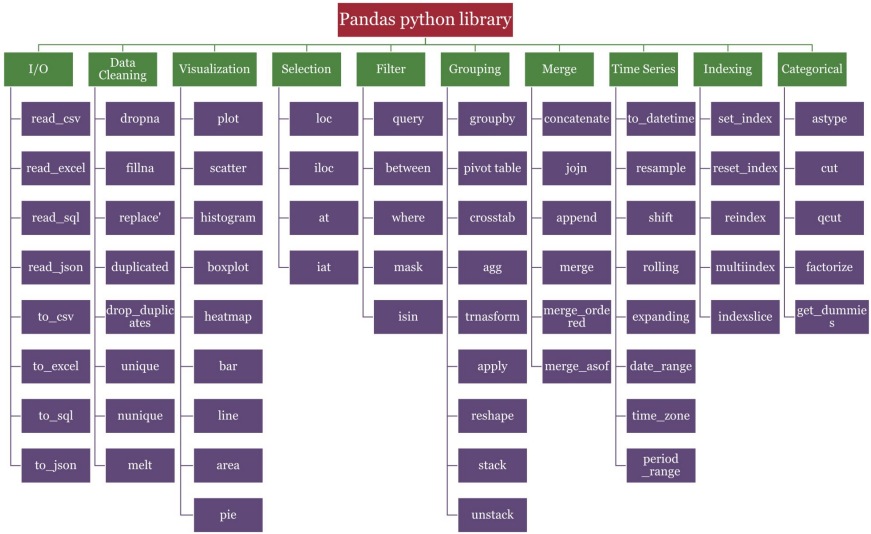
- pandas User Guide

libros:

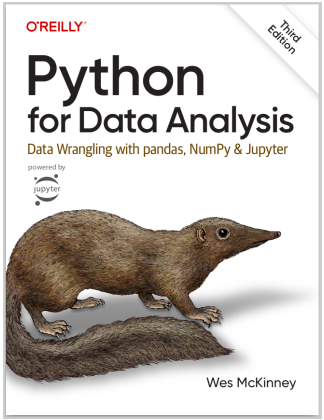
- "*Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*" Wes McKinney
- "*LEARNING pandas*" Stack Overflow contributors (gratis)

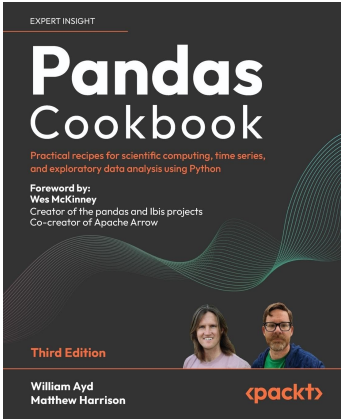
notebook 'cheat-sheet/chuleta':

- Pandas one-liners

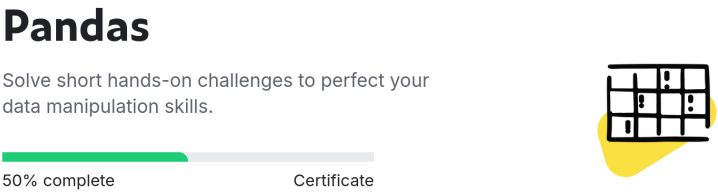

Pandas python library
I/O
read_csv
read_excel
read_sql
read_json
to_csv
to_excel
to_sql
to_json
Data Cleaning
dropna
fillna
replace'
duplicated
drop_duplicates
unique
nunique
melt
Visualization
plot
scatter
histogram
boxplot
heatmap
bar
line
area
pie
Selection
loc
iloc
at
iat
Filter
query
between
where
mask
isin
Grouping
groupby
pivot table
crosstab
agg
trnasform
apply
reshape
stack
unstack
Merge
concatenate
jojn
append
merge
merge_ordered
merge_asof
Time Series
to_datetime
resample
shift
rolling
expanding
date_range
time_zone
period_range
Indexing
set_index
reset_index
reindex
multiindex
indexslice
Categorical
astype
cut
qcut
factorize
get_dummies

Libros: "Python for Data Analysis" y "Pandas Cookbook"





BTW: You can obtain a free microcertificate in Pandas



from Kaggle Learn, which can then be added to your LinkedIn profile.

Visualization

- matplotlib
- seaborn
- plotly.express
- altair

Matplotlib

We shall use `matplotlib` via the `plt` interface

```
import matplotlib.pyplot as plt
```

We can make it look nicer by using a style sheet

```
plt.style.use('fivethirtyeight')
```

The basic `figure` container interface is inspired by the MATLAB syntax

```
plt.figure(figsize=(8, 6))
plt.plot(data["x123"], data["y1"], marker='o', linestyle='-', color="blue")
plt.plot(data["x123"], data["y2"], marker='o', linestyle='-', color="red")
plt.title("Figure plot")
plt.xlabel("X-axis label")
plt.ylabel("Y-axis label")
plt.xlim([6, 12])
plt.ylim([4, 10])
#plt.savefig('image.png', bbox_inches='tight')
plt.show();
```


Figure plot
Y-axis label
X-axis label
10
9
8
7
6
5
4
6
7
8
9
10
11
12

Within the `figure` object we can insert `axes` objects

```
fig, ax = plt.subplots(1, 2, figsize=(8, 6))
ax[0].plot(data["x123"], data["y1"], marker='o', linestyle='-', color="blue")
ax[1].plot(data["x123"], data["y2"], marker='o', linestyle='-', color="red")

ax[0].set_title("Axes object 0")
ax[0].set_xlabel("X-axis label")
ax[0].set_ylabel("Y-axis label")
ax[0].set_xlim([6, 12])
ax[0].set_ylim([4, 10])

ax[1].set_title("Axes object 1")
ax[1].set_xlabel("X-axis label")
ax[1].set_ylabel("Y-axis label")
ax[1].set_xlim([6, 12])
ax[1].set_ylim([4, 10])

plt.show();
```

each axes has its own coordinate system, data, and labels.


Axes object 0
Axes object 1
Y-axis label
Y-axis label
X-axis label
X-axis label
10
9
8
7
6
5
4
6
8
10
12

Note the different syntax between the `figure` and the `axes` objects

```
plt.title("Figure plot")
plt.xlabel("x-axis label")
plt.ylabel("y-axis label")
plt.xlim([6, 12])
plt.ylim([4, 10])

ax.set_title("Axes object methods")
ax.set_xlabel("x-axis label")
ax.set_ylabel("y-axis label")
ax.set_xlim([6, 12])
ax.set_ylim([4, 10])
```

Books: “Effective Visualization”, “Practical Charts”



